

SDN Batch 2

B.E/B.Tech. PRACTICAL END SEMESTER EXAMINATIONS

Sixth Semester

CCS365 - SOFTWARE DEFINED NETWORKS LABORATORY

(Regulation 2021) - SET 2 (FULL)

Time: 3 Hours | Max. Marks: 100

MARK ALLOCATION TABLE

Aim & Procedure	Commands & Program	Output & Results	Viva-Voce	Record	Total
40	20	20	10	10	100

DIRECT QUESTION MAPPING GUIDE

Check your exam question number and write the corresponding Master Experiment:

- If you get Q1, Q5, Q10, Q12, or Q18 -> Write **Master Experiment 1**
- If you get Q2, Q8, or Q19 -> Write **Master Experiment 2**
- If you get Q3 or Q16 -> Write **Master Experiment 3**
- If you get Q4 or Q13 -> Write **Master Experiment 4**
- If you get Q6 or Q14 -> Write **Master Experiment 5**
- If you get Q7, Q11, or Q15 -> Write **Master Experiment 6**
- If you get Q9, Q17, or Q20 -> Write **Master Experiment 7**

Master Experiment 1: Virtualbox/Mininet Environment

(For Questions: 1, 5, 10, 12, 18)

Aim & Procedure: To configure a virtual SDN lab using VirtualBox and the Mininet environment.

1. Download and install Oracle VirtualBox.

2. Download the official Mininet VM image (.ova file).
3. Import the Mininet VM into VirtualBox (`File -> Import Appliance`).
4. Configure the Network Adapter to "Host-only Adapter".
5. Start the VM, log in (user: `mininet` , pass: `mininet`), and verify the installation.

Commands & Program:

```
# Test the basic Mininet installation and connectivity
sudo mn --test pingall
```

Output & Results:

```
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (2/2 received)
```

Result: The VirtualBox and Mininet SDN environment was successfully set up and tested.

Master Experiment 2: Setup using Kathara

(For Questions: 2, 8, 19)

Aim & Procedure: To set up a virtual SDN lab using the Kathara network emulator.

1. Install Docker on the host machine (Kathara relies on containers).
2. Download and install the Kathara framework.
3. Create a project directory and a `lab.conf` file to define the network topology.
4. Run the Kathara start command to deploy the virtual network.
5. Connect to the virtual PC to test the network.

Commands & Program:

```
# Start the Kathara lab topology
kathara lstart

# Connect to a specific node (e.g., pc1)
kathara connect pc1
```

Output & Results:

```
[INFO] Starting lab...
[INFO] Deploying container pc1...
```

```
[INFO] Deploying container router1...  
[INFO] Lab started successfully.
```

Result: The Kathara SDN lab environment was successfully set up and started.

Master Experiment 3: Setup using GNS3

(For Questions: 3, 16)

Aim & Procedure: To set up a virtual SDN lab using the GNS3 network emulator.

1. Download and install GNS3 and the GNS3 VM.
2. Import the Open vSwitch (OVS) and an SDN Controller appliance.
3. Drag and drop the OVS, Controller, and two Virtual PCs (VPCs) into the workspace.
4. Connect the nodes using links and start the topology.
5. Assign IP addresses to the VPCs and perform a ping test.

Commands & Program:

```
# Configuration on VPC 1:  
ip 10.0.0.1 255.255.255.0  
  
# Configuration on VPC 2 & Ping test:  
ip 10.0.0.2 255.255.255.0  
ping 10.0.0.1
```

Output & Results:

```
Nodes successfully started.  
84 bytes from 10.0.0.1 icmp_seq=1 ttl=64 time=1.2ms
```

Result: The GNS3 virtual SDN lab was successfully set up and tested.

Master Experiment 4: Wireshark OpenFlow Capture

(For Questions: 4, 13)

Aim & Procedure: To create a simple Mininet topology with an SDN controller and use Wireshark to capture and visualize OpenFlow messages.

1. Start Mininet with a single switch and remote controller.
2. Open Wireshark and start capturing packets on the Loopback (`lo`) interface.

3. Type `openflow_v1` or `openflow_v4` in the Wireshark filter.
4. Use the Mininet CLI to ping between hosts and trigger OpenFlow traffic.

Commands & Program:

```
# Terminal 1: Start Mininet topology
sudo mn --topo single,3 --controller remote,ip=127.0.0.1

# Terminal 2: In Mininet CLI, trigger packets
mininet> h1 ping -c 3 h2
```

Output & Results:

In Wireshark, the following packets are captured and visualized:

1. OFPT_PACKET_IN (Switch sends unknown packet to Controller)
2. OFPT_FLOW_MOD (Controller instructs switch to add a rule)
3. OFPT_PACKET_OUT (Controller sends packet out of specific port)

Result: Mininet topology created and OpenFlow messages successfully captured via Wireshark.

Master Experiment 5: Northbound API (L2 Learning Switch)

(For Questions: 6, 14)

Aim & Procedure: To develop an SDN application using the Northbound API to program flow table rules on switches (L2 learning switch).

1. Start the Ryu controller running the `simple_switch_13` and REST API applications.
2. Start Mininet and connect it to the Ryu controller.
3. Perform a `pingall` to allow the controller to learn MAC addresses.
4. Use `curl` (Northbound REST API) to query the dynamically programmed flow rules.

Commands & Program:

```
# Terminal 1: Start Ryu with REST API
ryu-manager ryu.app.simple_switch_13 ryu.app.ofctl_rest

# Terminal 2: Use Northbound API to view flow rules on Switch 1
curl -X GET http://localhost:8080/stats/flow/1
```

Output & Results:

```
{
  "1": [
    {
      "actions": ["OUTPUT:2"],
      "match": {"in_port": 1, "dl_dst": "00:00:00:00:00:02"},
      "priority": 1
    }
  ]
}
```

Result: Flow table rules for the L2 learning switch were successfully programmed and retrieved via the Northbound API.

Master Experiment 6: End-to-end service with VNFs using vim-emu

(For Questions: 7, 11, 15)

Aim & Procedure: To create a simple end-to-end network service using two Virtual Network Functions (VNFs) deployed via vim-emu.

1. Install and start the `vim-emu` (NFV multi-PoP emulator) environment.
2. Start the default emulator topology to create a virtual datacenter.
3. Use the `vim-emu` CLI to deploy two VNFs (Docker containers) inside the datacenter.
4. Execute a ping command from VNF 1 to VNF 2 to verify network connectivity.

Commands & Program:

```
# Terminal 1: Start vim-emu topology
sudo python3 examples/default_single_dc_topology.py

# Terminal 2: Deploy VNFs and test
vim-emu compute start -d datacenter1 -n vnf1 -i ubuntu:trusty
vim-emu compute start -d datacenter1 -n vnf2 -i ubuntu:trusty
vim-emu compute exec -n vnf1 -- ping -c 3 vnf2
```

Output & Results:

```
Starting compute 'vnf1' in datacenter 'datacenter1'
Starting compute 'vnf2' in datacenter 'datacenter1'
64 bytes from vnf2: icmp_seq=1 ttl=64 time=0.051 ms
```

Result: An end-to-end network service with two VNFs was successfully created and verified.

Master Experiment 7: Install OSM & Orchestrate Network Service

(For Questions: 9, 17, 20)

Aim & Procedure: To install Open Source MANO (OSM) and onboard and orchestrate a network service.

1. Download the OSM installation script from the official ETSI repository.
2. Execute the installation script to deploy OSM containers.
3. Upload a Virtual Network Function Descriptor (VNFD) package via CLI.
4. Upload a Network Service Descriptor (NSD) package.
5. Create a Network Service (NS) Instance from the descriptors to orchestrate the service.

Commands & Program:

```
# Onboard Packages and Orchestrate via OSM CLI
osm vnfd-create hackfest_basic_vnf.tar.gz
osm nsd-create hackfest_basic_ns.tar.gz
osm ns-create --ns_name test_ns --nsd_name hackfest_basic-ns --vim_account
my_vim
```

Output & Results:

```
VNFD successfully onboarded.
NSD successfully onboarded.
Network Service Instance 'test_ns' created and is in 'READY' state.
```

Result: OSM was successfully installed, and the network service was orchestrated.
